PKMon 校园 LBS 收集游戏 - 项目设计草案

一、项目概述 (Project Overview)

1. 核心愿景

PKMon 是一款结合地理位置服务（LBS）与轻量化知识问答的校园探索应用。项目旨在将日常校园生活（如上课、自习、运动）转化为收集专属校园精灵的乐趣，鼓励学生探索校园并了解校园文化。

精灵的呈现与收集体验参考 Pokémon GO：精灵以自身形象直接出现在真实地图上，点击后进入独立的抓捕界面，捕获后点亮图鉴。

考虑到本项目是课程周期内的 8 人小组作业，整体定位为轻量实现：采用手机 App（Flutter）+ 原生默认 2D 地图展示，不做 3D 模型与 AR 实景合成，以“校园特色问答”作为核心抓捕机制。

2. 核心玩法 (Core Gameplay)

- **探索与发现**：玩家在真实的校园地图上移动，地图上会实时出现可捕捉的 PKMon。
- **多维触发机制**：精灵的出现不仅依赖“经纬度”，还可能与“时间（如凌晨）”、“驻留（自习时长）”和“轨迹（运动路线）”深度绑定（具体触发条件待定）。
- **问答抓捕**：遭遇精灵后，需正确回答一道与该精灵特性或出现地点相关的校园知识题方可捕获。
- **图鉴与社交**：点亮校园图鉴，并在校园排行榜上展示收集进度（排行榜等社交功能优先级待定）。

3. 交互主流程

整个游戏是一个闭环，玩家大部分时间停留在**地图主界面**：

> 地图主界面（移动 / 探索） → 触发条件满足，精灵出现在地图上 → 点击精灵 → **进入全屏精灵抓捕界面** → 答对校园知识题 → 捕获成功动效 → **返回地图主界面** → 在「图鉴」与「我的精灵」中可查看这只精灵

二、界面结构（页面清单）

#	页面	说明
1	**地图主界面**	默认首页，交互形式参考高德／腾讯地图：全屏 2D 地图、玩家位置居中、以玩家为中心的半透明“感知半径”、地图上的精灵、底部导航栏（地图 / 图鉴 / 我的精灵 / 我的）、悬浮按钮（回到当前位置、开启任务）。
2	**精灵抓捕界面**	点击地图上的精灵后全屏覆盖进入（布局参考 Pokémon GO）：精灵立绘居中 + 出场动效，下方为题目与选项。答对播放捕获成功动效后返回地图；答错显示精灵逃跑／冷却提示。**这是与地图界面并列的核心界面。**
3	**图鉴 (PKDex)**	网格列表。已捕获显示立绘，未捕获显示剪影，顶部为收集进度条，按编号／院系／属性分类（具体维度待定）。点击进入精灵详情。
4	**我的精灵（背包）**	玩家已捕获精灵的列表，可按捕获时间／地点／稀有度排序，与图鉴的区别是这里只展示“我拥有的”，并支持查看重复数量。
5	**精灵详情页**	立绘、属性、稀有度、捕获时间与捕获地点（可在地图上回显捕获点）。

#	页面	说明
6	**个人中心（我的）**	用户信息、收集进度统计、权限与任务模式设置、好友入口。
7	**好友与交换界面**	（后续规划，见 3.6）好友列表、交换请求、双人交换界面。

三、核心功能模块 (Core Modules)

1. 基础 LBS 地图展示

- 接入高德或腾讯地图的原生默认 2D 地图作为底图，降低美术与渲染开发成本。
- 实时同步玩家 GPS 坐标，并在地图上渲染以玩家为中心的半透明可视“感知半径”（如 50 米，具体数值待定）。
- 精灵以自身形象（2D Sprite）直接出现在地图对应的经纬度上，而不是通用的图钉（Marker）；不同稀有度可用不同的出场动效或光环区分（具体表现待定）。

2. 状态机与多维触发系统

- **空间围栏触发（Geo-fencing）**：玩家进入预设的多边形区域触发精灵（举例：进入理科楼区域触发“理科水怪”）。
- **时间/驻留触发（Dwell Time）**：玩家在特定区域连续心跳打卡超过指定时间触发（举例：在图书馆停留 2 小时解锁“爆肝学霸”，具体时长与判定规则待定）。
- **轨迹匹配触发（Trajectory）**：玩家开启特定任务后，系统记录并分析其运动轨迹匹配触发（举例：绕未名湖跑圈触发限定精灵，具体轨迹路线待定）。

3. 校园问答抓捕机制

当玩家点击地图上的精灵时，进入全屏 2D 抓捕界面，布局参考 Pokémon GO：精灵立绘居中，下方为题目与选项。

- **题库绑定**：每只 PKMon 绑定一个特定的标签库。例如遭遇“图书馆精灵”时，系统从“图书馆规则与历史”题库中随机抽取题目。
- **交互流程**：弹出带有精灵立绘的对话框进行提问（举例：“你知道老版校徽是谁设计的吗？”或“理科教学楼几点锁门？”）。答对即可收入图鉴（配合捕获成功动效），答错则精灵逃跑或进入冷却时间（具体惩罚机制待定）。

4. 精灵图鉴系统 (PKDex)

- **数据持久化**：记录玩家已捕获精灵的基础信息、捕获时间、捕获地点、捕获次数及稀有度。
- **分类展示**：（可以使用新形象如学院吉祥物等，也可以考虑直接宝可梦的形象）参考 Pokédex，按编号 / 院系 / 属性等维度分类，已捕获的精灵显示立绘，未捕获的显示剪影，并提供收集进度条（具体分类维度待定）。

5. 我的精灵（背包）

- 只展示当前玩家拥有的精灵，是图鉴的“实体”版本：图鉴记录“是否见过／捕获过”，背包记录“我手上有几只”。

- 支持按捕获时间、捕获地点、稀有度排序与筛选，为后续的交换功能提供入口。

6. 好友与精灵交换（后续规划，优先级 V2）

- **前提**：好友系统（通过用户 ID / 二维码添加）。
- **流程**：发起交换 → 双方各选一只精灵 → 双方确认 → 服务端校验两只精灵当前归属 → 原子性地互换所有权 → 双方图鉴与背包同步更新。
- **界面**：好友列表、交换请求通知、双人交换界面（左右各一只精灵 + 确认按钮 + 交换动效）。
- **技术要点**：必须用数据库事务 + 行锁 / 乐观锁，防止同一只精灵被并发交换；可加限制条件（稀有度对等、每日次数上限）。
- **定位**：核心闭环跑通后再做，属于加分项。

四、技术选型与架构设计

模块	推荐技术栈	选型理由与说明
前端/客户端	Flutter（手机 App，iOS/Android 双端）	驻留计时与轨迹记录都需要长时间后台定位，独立 App 的保活策略比小程序宽松得多（小程序的后台定位接口仅对特定类目开放，本项目基本申请不到）。一套代码跨双端，交互流畅度高。
后端框架	Spring Boot (Java/Kotlin)	团队协作的业界标准，生态丰富，易于实现 RESTful API 和复杂业务逻辑拆分。
数据库	PostgreSQL (+ PostGIS) + Redis (Geo)	PostgreSQL 存储用户、题库、图鉴与**轨迹历史**（需要 PostGIS 扩展做空间查询）。 **Redis Geo** 负责**实时**部分：基于 GeoHash 以毫秒级延迟算出“玩家附近有哪些精灵”，并缓存玩家的当前任务状态（所在围栏、驻留起始时间等）。
地图 SDK	高德地图 / 腾讯地图 API	提供稳定的高精度经纬度获取、逆地理编码、Wi-Fi 辅助定位，以及原生地图展示和覆盖物（Marker）API。

五、8 人团队敏捷分工方案

1. **项目经理 & 产品 (PM) - 1 人**

- 定义 PKMon 的属性、稀有度、题库内容整理和刷新规则。
- 撰写 API 接口文档规范，控制项目进度和需求边界。
- 负责最终展示 PPT 和 Demo 演示路径设计。

2. **UI/UX 与美术设计 - 1 人**

- 负责 App 整体界面原型设计（地图主界面、抓捕界面、图鉴、背包）。
- 产出 2D 精灵原画、地图上的精灵形象、以及问答界面的 UI 元素（精灵形象具体风格待定，可借助 AI 工具辅助）。

3. **大前端开发 - 2 人**

- **前端 A**：专攻地图 SDK 接入、原生默认地图展示、GPS 坐标实时更新、精灵在地图上的渲染及出场动效，以及**后台定位与任务模式**的实现。

- **前端 B**：专攻问答抓捕界面的交互开发、图鉴与背包页面展示和网络请求封装。

4. **后端与核心算法 - 3 人**

- **后端 A（架构与数据）**：搭建工程，设计数据库表结构，实现 Redis Geo 坐标管理和题库随机抽取逻辑。
- **后端 B（业务逻辑）**：编写用户系统、图鉴背包系统、问答校验结算的 API。
- **后端 C（LBS 算法）**：核心攻坚，专门负责编写"绕湖轨迹匹配"和"驻留时间判定"的统计算法。

5. **QA 测试与运维 - 1 人**

- 统一部署测试环境与数据库。
- 使用虚拟定位工具与**轨迹回放工具**编写自动化测试脚本，测试各类坐标触发机制与问答结算流程。

六、核心技术难点与实现思路

1. 位置与轨迹的采集 —— 本项目最关键、也最容易翻车的一环

> **核心认知**：难点不在于"拿到一次坐标"（调用地图 SDK 就有），而在于**在 App 退到后台、屏幕熄灭、用户不主动操作的情况下，持续且可靠地拿到坐标**。这是贯穿驻留判定与轨迹判定的共同底座，也是预算的大头。

1.1 后台保活（最大的坑）

- **Android**：**前台服务（Foreground Service）+ 常驻通知栏**是唯一可靠的方案。需要申请 `ACCESS_BACKGROUND_LOCATION`（Android 10 起需单独授权，且必须跳转到系统设置页由用户手动选择"始终允许"）；Android 14 起前台服务还必须声明 `foregroundServiceType="location"`。
  - 真正的难点是**厂商 ROM**：小米/华为/OPPO/vivo 的自启动与省电策略会杀掉后台，必须引导用户手动加白名单。
- **iOS**：需要 `UIBackgroundModes: location` 与 `allowsBackgroundLocationUpdates`，并要求"始终允许"授权；系统会定期弹窗二次确认，也会为省电自动降低定位频率；App 被用户手动划掉后不会自动重启。
- **工程建议**：**演示与验收锁定 Android 真机**。iOS 后台定位的坑多、耗时不可控，不适合压在课程工期里。

1.2 精度与漂移

- 室外开阔地约 5~10 米，楼宇间 20~50 米，**室内基本失效**。
- ⚠️ 这一条直接冲击"在图书馆驻留 2 小时"：图书馆里 GPS 拿不到可信坐标，玩家会看到自己在地图上飘出围栏。
- **对策**：
  1. 丢弃精度半径过大的点（如 `accuracy > 50 米` 直接不接受）；
  2. 围栏加缓冲带，进入／离开都要**连续 2 次命中**才确认状态切换，避免在边界反复横跳（geo-fence flapping）；
  3. 室内场景借助地图 SDK 的 Wi-Fi 辅助定位，或把"驻留判定"从"精确坐标"放宽为"低频坐标 + 较大围栏"。
  4. 轨迹判定里用"速度合理性"过滤漂移点（如瞬间移动 500 米视为异常）。

1.3 电量与流量

- 固定每 5 秒上报一次太耗电，用户会关掉 App。

- **自适应采样**：静止 60 秒 / 步行 10 秒 / 跑步 5 秒（依据 Android Activity Recognition 或 iOS CoreMotion 判断运动状态）。
- **批量上传**：攒够 10 个点或满 60 秒再打包上报，减少请求数；轨迹用 Douglas-Peucker 算法抽稀。
- 数据量其实很小：跑步 1 小时按 5 秒采样约 720 个点、30KB 左右，存储不构成瓶颈。

1.4 权限降级

- Android 12+ 用户可能只授予"大致位置"（精度 1~3 公里），游戏会直接不可用 → 必须做引导文案与降级处理，而不是卡死。
- iOS 14+ 有"仅一次"选项；首次拒绝后要能引导去系统设置页重新授权。

1.5 强烈建议：把"全天候后台追踪"降级为"任务模式"

不要试图 24 小时追踪用户。改为：

- 玩家主动开启"跑步模式"／"自习模式"，此时才启动前台服务与常驻通知，任务结束后自动停止；
- 或者由前端在检测到进入围栏时**弹通知询问**"是否开始打卡"，用户确认后再开启。
- 收益：技术风险大幅下降（不必正面对抗系统杀进程）、更省电、对用户透明（隐私友好）、**答辩演示时完全可控**。Pokémon GO 本身也是"打开 App 才追踪"。

2. 触发判定（全部放在服务端）

> **核心原则：客户端只负责采集与上报，所有"是否触发"的判定都在服务端完成。** 好处：判定逻辑集中、可以单元测试、改规则不用发版、位置数据可回放重算，客户端也不需要被信任。

2.1 驻留时长判定

应用退到后台后容易被系统"杀进程"，无法保持长连接持续上报。

- **心跳机制（Heartbeat）**：前端进入"图书馆"多边形围栏后，触发驻留任务，每隔 3-5 分钟向后端发送一次带有当前 GPS 的心跳包。
- **容错处理**：后端记录该玩家的 `last_heartbeat_time`。考虑到网络波动或短暂离开（如去洗手间），允许最多错失 2 次心跳。如果心跳断连超过设定阈值（如 15 分钟），或最新心跳的 GPS 已经离开围栏，则驻留任务中断，计时清零。

2.2 轨迹匹配判定（轨迹打卡点待定）

仅对比起点和终点无法区分"真的绕湖跑了一圈"和"在原地来回走动"，需要引入轨迹点过滤与围栏顺序切割：

1. **数据采集与去噪**：前端每隔一定时间（如 5 秒）上报一次 GPS 坐标。后端接收后，过滤掉漂移过大（如瞬间移动 500 米）的异常点。
2. **轨迹切割**：在后端将"未名湖"及周边道路划分为 4 个连续的隐形地理围栏（A、B、C、D）。
3. **状态机判定**：玩家的数据流必须在设定的时间窗口内（如 30 分钟），按顺序或逆序依次产生穿过 A -> B -> C -> D -> A 的打卡记录，形成闭环后即判定为"跑圈成功"。

3. 与精灵特性绑定的问答系统

- **表结构设计**：设计两张核心表 `PKMon_Dict`（精灵字典）和 `Question_Bank`（题库）。通过 `tag_id`（如"历史"、"理科"、"运动"）进行多对多关联。
- **抽取逻辑**：当玩家触发抓捕请求时，后端根据该精灵的 `tag_id` 从题库中随机抽取一道状态为"启用"的题目返回给前端。

- **答案校验**：前端只接收题目与选项，不下发正确答案；玩家提交选项后由后端比对并返回抓捕结果。

4. 如何在不真的走一圈的情况下测试（很实际的问题）

QA 只有 1 人，不可能天天在校园里跑步来验证触发逻辑。必须让判定逻辑**可脱离真实定位来测**：

- **解耦位置来源**：前端把定位抽象成一个接口（`LocationProvider`），线上用真实 GPS，调试时用**轨迹回放**（加载 GPX 文件，按真实时间间隔注入坐标）。
- **服务端判定写成纯函数**：输入一串坐标点与时间戳，输出判定结果。这样单元测试可以直接喂构造好的轨迹数组，秒级验证"绕圈／不绕圈／反着绕"等边界情况。
- 这也是把判定放在服务端（而非客户端）的又一个理由。

七、开工前置准备：权限、账号与合规

> 这些事不写代码，但**在关键路径上**，且有等待审核的环节，建议开工第一周就启动。

1. 系统定位权限（必修，不是可选项）

- **Android**：在 `AndroidManifest.xml` 声明 `ACCESS_FINE_LOCATION`（精确定位）与 `ACCESS_COARSE_LOCATION`（大致定位）是基础；后台定位另需 `ACCESS_BACKGROUND_LOCATION`。Android 6.0 起为**运行时权限**，必须在代码中动态申请并由用户弹窗同意；Android 10 起后台定位**必须单独申请且要跳转系统设置页**，无法与普通权限一起弹窗；Android 12 起用户可在"精确／大致"之间二选一。
- **iOS**：`Info.plist` 中**必须**填写 `NSLocationWhenInUseUsageDescription` / `NSLocationAlwaysAndWhenInUseUsageDescription`，缺失会直接崩溃；iOS 13 起有"仅允许一次"选项。
- **用户随时可以拒绝或撤销**，所以必须有权限被拒后的降级路径与引导文案，而不是卡死在启动页。

2. 地图 SDK 账号与 Key（最容易被低估的"行政工作"）

- 需注册**高德开放平台／腾讯位置服务**的开发者账号，并完成**实名认证**（个人开发者需身份证，审核需要时间）。
- 创建应用时必须绑定 **Android 包名 + 签名 SHA1**（iOS 绑 Bundle ID）。**注意 debug 与 release 签名不同，需要分别绑定两个 Key**；Flutter 项目可用 `cd android && ./gradlew signingReport` 获取签名信息。
- Key 按平台分类且**不可混用**：Android Key、iOS Key、**Web 服务 Key（后端调用，配额独立）**。前端与后端要各申请各的。
- 个人开发者的免费调用配额有限（日调用量、并发数），课程项目通常够用，但需留意。
- **建议指定 1 人在开工第一周完成注册、实名与建应用**，否则前端会卡在"拿不到 Key"上。

3. 合规注意事项

- 位置信息属于**敏感个人信息**，《个人信息保护法》要求单独告知与同意，并配套隐私政策说明收集目的、范围与存储方式。
- **中国大陆境内展示地图必须使用有测绘资质的厂商 SDK**（高德／腾讯／百度），不要自行下载地图瓦片渲染。注意 Google Maps 在国内不可用。
- 本项目作为课程作业**不上架应用商店**，可基本豁免商店审核与 App 备案等流程；建议在文档与 App 内注明"仅用于课程学习，非商业用途"。

八、当前待定事项

- 精灵刷新与捕获规则：全局唯一（先到先得）还是每人独立一份？冷却时间？稀有度如何划分？
- 精灵形态最终是 2D 立绘还是 3D 模型（本文档按 2D 立绘编写）。
- 各类触发的具体数值：感知半径、驻留时长、轨迹时间窗口、答错惩罚。
- 图鉴的分类维度、社交／排行榜功能是否纳入本次范围。
- 完整排期与里程碑。